

Automation Framework For SDET

Part 2: Execution, CI/CD & Best Practices

★ Topics 13 to 24 — Execution to DevOps ★



Kalpesh Jain

SDET @ Algebrik AI | Mentor | 42K+ Community ■

Bengaluru, Karnataka, India

[linkedin.com/in/kalpeshnjain09](https://www.linkedin.com/in/kalpeshnjain09)

JSPM University Pune

Screenshots on Failure	Retry Mechanism	Listeners	Assertions Best Practices
Wait Strategies	Parallel Execution	Cross-Browser Execution	CI/CD Integration
GitHub Actions	Jenkins	Docker	Framework Best Practices

Introduction

This is **Part 2** of the Automation Framework For SDET Complete Guide. It covers **Topics 13–24** — execution strategies, CI/CD integration, and best practices asked in senior SDET and product company interviews.

Complete Part 1 (Topics 01–12) before studying these topics.

Topic Group	What You Will Learn
Screenshots, Retry & Listeners	Failure handling & event-driven automation
Assertions & Wait Strategies	Validation patterns & synchronization
Parallel & Cross-Browser	Execution optimization across browsers
CI/CD: GitHub Actions & Jenkins	Automated pipeline integration
Docker & Best Practices	Containerization & framework standards

Table of Contents

PART 2 — EXECUTION, CI/CD & BEST PRACTICES (Topics 13–24)

- 13. Screenshots on Failure ★
- 14. Retry Mechanism ★
- 15. Listeners ★
- 16. Assertions Best Practices ★
- 17. Wait Strategies ★★★
- 18. Parallel Execution ★★★
- 19. Cross-Browser Execution ★★★
- 20. CI/CD Integration ★★★
- 21. GitHub Actions ★★
- 22. Jenkins ★★★
- 23. Docker ★★
- 24. Framework Best Practices ★★★

SCREENSHOTS ON FAILURE

13. Screenshots on Failure ★

► What are Screenshots on Failure?

What is it: Screenshots on Failure is a framework feature that automatically captures the application screen whenever a test case fails. This helps QA engineers and developers quickly identify the application state at the moment of failure.

Key Components:

- Automatic Screenshot Capture
- Failure Detection
- Screenshot Storage
- Report Attachment
- Timestamp-based Naming

How YOU should answer: Screenshots on Failure is an automation framework feature that automatically captures the browser screen whenever a test fails, making debugging faster and providing visual evidence of failures.

Example

```
Login Test
↓
Click Login
↓
Dashboard Not Loaded
↓
Capture Screenshot
↓
```

► Why Screenshots are Important

What is it: Screenshots provide visual proof of failures and reduce debugging time.

Key Components:

- Faster Debugging
- Visual Evidence
- Easier Defect Reporting
- Better Communication
- Historical Reference

How YOU should answer: Screenshots help understand the exact UI state when a test fails, reducing the need to reproduce issues manually.

Example

```

Button Missing
↓
Screenshot Captured
↓
Developer Reviews UI

```

Common Mistakes

- ✗ Capturing screenshots for every step.
- ✗ Overwriting screenshots.
- ✗ Not capturing screenshots for failures.
- ✗ Capturing screenshot after browser closes.

Expected Interview Questions

- What are Screenshots on Failure?
- Why are they important?
- Why are screenshots useful?
- Which types of screenshots have you used?
- How do you name screenshots?

★ Final Interview Gold

1. - Screenshots on Failure
2. - Screenshot Capture Flow
3. - Screenshot Naming Strategy
4. - Report Integration
5. - Enterprise Screenshot Handling
6. - Screenshot Best Practices
7. Retry Mechanism
8. The next topic is:

RETRY MECHANISM

14. Retry Mechanism ★

► What is Retry Mechanism?

What is it: Retry Mechanism is a framework feature that automatically re-executes a failed test case a predefined number of times before marking it as failed. It helps reduce failures caused by temporary issues such as network delays, slow page loads, or unstable environments.

Key Components:

- Automatic Retry
- Retry Count
- Failure Detection
- Final Result
- Retry Configuration

How YOU should answer: Retry Mechanism automatically reruns failed test cases a configured number of times to handle temporary failures and reduce flaky test results.

Example

Login Test

↓

Attempt 1

↓

Network Timeout

↓

Retry

↓

► Why Retry Mechanism is Important

What is it: Retry helps improve automation stability by reducing failures caused by environmental issues rather than application defects.

Key Components:

- Reduce Flaky Failures
- Stable CI/CD
- Reliable Regression
- Less Manual Re-execution
- Better Success Rate

How YOU should answer: Retry improves automation reliability by automatically handling temporary failures without requiring manual execution.

Example

500 Regression Tests

↓

5 Temporary Failures

↓

Retry

↓

4 Tests Pass

↓

Common Mistakes

- ✗ Using unlimited retries.
- ✗ Retrying genuine application bugs.
- ✗ Assuming Retry fixes application defects.
- ✗ Using Retry instead of fixing flaky tests.

Expected Interview Questions

- What is Retry Mechanism?
- Why do we use Retry?
- Why is Retry important?
- Retry vs Re-run?
- What causes flaky tests?

★ Final Interview Gold

1. - Retry Mechanism
2. - Retry vs Re-run
3. - Flaky Tests
4. - Retry Execution Flow
5. - Playwright Retry Configuration
6. - Enterprise Retry Strategy
7. - Retry Best Practices
8. Listeners

LISTENERS

15. Listeners ★

► What are Listeners?

What is it: Listeners are framework components that automatically listen to test execution events and perform predefined actions whenever those events occur. Instead of manually writing code for logging, screenshots, reporting, or notifications inside every test, Listeners execute automatically when specific ev

Key Components:

- Test Start Event
- Test Pass Event
- Test Failure Event
- Test Skip Event
- Suite Start
- Suite End

How YOU should answer: Listeners are event-driven components that automatically execute predefined actions during different stages of test execution, such as logging, screenshot capture, reporting, and notifications.

Example

Test Started

↓

Listener Triggered

↓

Log Test Start

↓

Execute Test

↓

► Why Listeners are Important

What is it: Listeners automate repetitive framework tasks without modifying test scripts.

Key Components:

- Automation
- Reusability
- Centralized Event Handling
- Cleaner Test Code
- Better Maintainability

How YOU should answer: Listeners improve framework maintainability by centralizing actions that should happen automatically during test execution.

Example

100 Test Cases

↓

One Listener

↓

Automatic Logging

↓

Automatic Screenshot

↓

Common Mistakes

- ✗ Writing listener logic inside test cases.
- ✗ Mixing business logic with listener logic.
- ✗ Duplicate event handling code in every test.
- ✗ Heavy business logic inside listeners.

Expected Interview Questions

- What are Listeners?
- Why do we use Listeners?
- Why are Listeners important?
- Which listener events have you used?
- Listener vs Hooks?

★ Final Interview Gold

1. - Listeners
2. - Listener Events
3. - Listener Execution Flow
4. - Listener vs Hooks
5. - Enterprise Listener Implementation
6. - Report & Screenshot Integration
7. - Listener Best Practices
8. Assertions Best Practices

ASSERTIONS BEST PRACTICES

16. Assertions Best Practices ★

► What are Assertions?

What is it: Assertions are validation statements that verify whether the actual application behavior matches the expected behavior. If an assertion passes, the test continues successfully. If it fails, the test is marked as failed. Assertions are the core of every automated test because they determine whether t

Key Components:

- Expected Result
- Actual Result
- Validation
- Pass/Fail Decision
- Error Message

How YOU should answer: Assertions are validation checkpoints used to compare expected and actual results. They determine whether a test case passes or fails.

Example

Login

↓

```
Dashboard Visible?
```

```
↓
```

```
Yes
```

```
↓
```

```
Assertion Passed
```

► Why Assertions are Important

What is it: Assertions verify that application functionality behaves as expected. Without assertions, automation only performs actions and cannot determine whether the test actually succeeded.

Key Components:

- Validation
- Reliability
- Test Accuracy
- Bug Detection
- Automation Confidence

How YOU should answer: Assertions ensure automation validates application behavior instead of simply executing actions.

```
Example
```

```
Click Login
```

```
↓
```

```
Dashboard Loaded?
```

```
↓
```

```
Assertion
```

```
↓
```

```
Pass
```

Common Mistakes

- ✗ No assertions in test cases.
- ✗ Incorrect expected values.
- ✗ Assuming clicking a button means success.
- ✗ Too many assertions in one test.

Expected Interview Questions

- What are Assertions?
- Why do we use Assertions?
- Why are Assertions important?
- Which assertions have you used?
- Hard Assert vs Soft Assert?

★ Final Interview Gold

1. - Assertions
2. - Hard vs Soft Assertions
3. - Playwright Assertions
4. - Auto-waiting Assertions
5. - Assertion Execution Flow
6. - Enterprise Assertion Strategy
7. - Assertion Best Practices
8. Wait Strategies

WAIT STRATEGIES

17. Wait Strategies ★

► What are Wait Strategies?

What is it: Wait Strategies are synchronization techniques used to ensure automation interacts with the application only after it reaches the expected state. Modern web applications load elements asynchronously, making waits essential for reliable automation. Without proper waits, automation becomes flaky because

Key Components:

- Synchronization
- Element Readiness
- Page Loading
- Stability
- Auto Waiting

How YOU should answer: Wait Strategies synchronize automation execution with application behavior, ensuring elements are ready before interactions occur.

Example

Open Login Page

↓

Login Button Still Loading

↓

Wait

↓

Button Visible

↓

► Why Wait Strategies are Important

What is it: Wait Strategies improve automation reliability by handling dynamic application behavior.

Key Components:

- Synchronization
- Stability
- Less Flaky Tests
- Better Performance
- Reliable Execution

How YOU should answer: Wait Strategies prevent synchronization issues by ensuring the application is ready before automation performs actions.

Example

500 Regression Tests

↓

Without Waits

↓

40 Failed

With Proper Waits

↓

Common Mistakes

- ✗ Using Thread.sleep() everywhere.
- ✗ Clicking elements immediately after page load.
- ✗ Replacing synchronization with fixed delays.
- ✗ No |

Expected Interview Questions

- What are Wait Strategies?
- Why do we need waits?
- Why are waits important?
- Why do modern applications require waits?
- How does Playwright Auto-Wait work?

★ Final Interview Gold

1. - Wait Strategies
2. - Playwright Auto-Wait
3. - Explicit vs Implicit Wait
4. - Types of Waits

- 5. - Enterprise Wait Strategy
- 6. - Wait Execution Flow
- 7. - Wait Best Practices
- 8. Parallel Execution

PARALLEL EXECUTION

18. Parallel Execution ★

► What is Parallel Execution?

What is it: Parallel Execution is the process of running multiple test cases simultaneously across multiple browser instances, workers, or machines instead of executing them one after another. It significantly reduces execution time and makes large regression suites practical for CI/CD pipelines.

Key Components:

- Workers
- Multiple Browser Instances
- Independent Test Execution
- Resource Utilization
- Faster Execution

How YOU should answer: Parallel Execution runs multiple test cases simultaneously using multiple workers or browser instances, reducing execution time and improving CI/CD efficiency.

Example

100 Test Cases

↓

4 Workers

↓

25 Tests Per Worker

↓

All Execute Simultaneously

↓

► Why Parallel Execution is Important

What is it: Parallel execution dramatically reduces regression execution time.

Key Components:

- Faster Regression
- Better Resource Usage
- CI/CD Friendly

- Faster Feedback
- Scalability

Example

Sequential

200 Tests

↓

200 Minutes

Parallel

8 Workers

↓

Common Mistakes

- ✗ Shared test data between workers.
- ✗ Test dependencies.

Expected Interview Questions

- What is Parallel Execution?
- Why do we use it?
- Why is Parallel Execution important?
- Sequential vs Parallel Execution?
- How does Playwright execute tests in parallel?

★ Final Interview Gold

1. - Parallel Execution
2. - Sequential vs Parallel
3. - Playwright Workers
4. - Parallel Execution Flow
5. - Enterprise Parallel Strategy
6. - Parallel Challenges
7. - Parallel Best Practices
8. Cross-Browser Execution

CROSS-BROWSER EXECUTION

19. Cross-Browser Execution ★

► What is Cross-Browser Execution?

What is it: Cross-Browser Execution is the process of running the same automation test suite across multiple browsers to ensure the application behaves consistently. Different browsers use different rendering engines, JavaScript engines, and implementations, which can lead to browser-specific issues.

Key Components:

- Browser Compatibility
- Multiple Browser Testing
- Rendering Validation
- Consistent User Experience
- Automated Browser Execution

How YOU should answer: Cross-Browser Execution verifies that the application behaves consistently across different browsers by executing the same automation suite on multiple browser engines.

Example

Regression Suite

↓

Chrome

Firefox

Edge

↓

Compare Results

↓

► Why Cross-Browser Execution is Important

What is it: Users access applications using different browsers, operating systems, and devices. Cross-browser execution ensures a consistent experience for all users.

Key Components:

- Browser Compatibility
- Better User Experience
- Wider Test Coverage
- Early Defect Detection
- Production Confidence

How YOU should answer: Cross-Browser Execution ensures that browser-specific issues are identified before production, improving application quality and user experience.

Example

Chrome

↓

Pass

Firefox

↓

CSS Issue

Edge

↓

Common Mistakes

- ✗ Testing only Chrome.
- ✗ Assuming all browsers behave identically.
- ✗ Ignoring less-used browsers.
- ✗ Running every browser sequentially.

Expected Interview Questions

- What is Cross-Browser Execution?
- Why do we need it?
- Why is Cross-Browser Testing important?
- What browser-specific issues have you faced?
- Which browsers does Playwright support?

★ Final Interview Gold

1. - Cross-Browser Execution
2. - Playwright Browser Projects
3. - Supported Browsers
4. - Browser Compatibility Challenges
5. - Enterprise Cross-Browser Strategy
6. - Cross-Browser Execution Flow
7. - Cross-Browser Best Practices
8. CI/CD Integration

CI/CD INTEGRATION

20. CI/CD Integration ★

► What is CI/CD?

What is it: CI/CD (Continuous Integration and Continuous Delivery/Deployment) is a software development practice that automates building, testing, and deploying applications whenever code changes are committed. Automation testing is integrated into the pipeline so every code change is validated automatically before deployment.

Key Components:

- Source Control (Git)
- Build Process
- Automated Testing
- Reporting
- Deployment
- Monitoring

How YOU should answer: CI/CD is a DevOps practice that automates code integration, testing, and deployment. Automation tests are executed automatically whenever code changes are pushed, ensuring faster feedback and higher software quality.

Example

Developer Pushes Code

↓

Pipeline Starts

↓

Build Application

↓

Run Automation Tests

↓

► Why CI/CD Integration is Important

What is it: CI/CD enables rapid feedback by executing automation immediately after code changes.

Key Components:

- Faster Feedback
- Continuous Testing
- Early Defect Detection
- Reduced Manual Effort
- Faster Releases

How YOU should answer: Integrating automation with CI/CD ensures every code change is automatically validated before deployment, reducing production defects and enabling faster releases.

Example

Developer Commit

↓

5 Minutes Later

↓

Automation Results Ready

Common Mistakes

- ✗ Running automation manually after every build.

- ✗ No automated quality gates.
- ✗ Running regression only before release.
- ✗ Deploying without automated testing.

Expected Interview Questions

- What is CI/CD?
- Why do we integrate automation with CI/CD?
- Why is CI/CD important?
- CI vs CD?
- Difference between Continuous Delivery and Continuous Deployment?

★ Final Interview Gold

1. - CI/CD
2. - CI vs CD
3. - CI/CD Pipeline Flow
4. - Framework Integration
5. - Enterprise Pipeline
6. - CI/CD Tools
7. - CI/CD Best Practices
8. GitHub Actions

GITHUB ACTIONS

21. GitHub Actions ★

► What is GitHub Actions?

What is it: GitHub Actions is GitHub's built-in CI/CD platform that automates software workflows such as building, testing, and deploying applications directly from a GitHub repository. Whenever code is pushed or a pull request is created, GitHub Actions can automatically execute automation tests without requiring manual intervention.

Key Components:

- Workflow
- Event Trigger
- Runner
- Jobs
- Steps
- Actions

How YOU should answer: GitHub Actions is GitHub's built-in CI/CD automation platform that executes workflows whenever configured events occur, such as code pushes or pull requests.

Example

Developer Pushes Code

↓

GitHub Actions Triggered

↓

Run Playwright Tests

↓

Generate HTML Report

↓

► Why GitHub Actions is Important

What is it: GitHub Actions automates repetitive development and testing tasks, ensuring every code change is validated automatically.

Key Components:

- Automation
- Continuous Testing
- Faster Feedback
- Cloud Execution
- Easy Integration

How YOU should answer: GitHub Actions provides automated CI/CD directly inside GitHub, enabling continuous testing and faster feedback without external tools.

Example

Developer Commit

↓

Workflow Runs

↓

Smoke Tests

↓

Results Ready

Common Mistakes

- ✗ Running workflows manually every time.

Expected Interview Questions

- What is GitHub Actions?
- Why do we use GitHub Actions?
- Why is GitHub Actions important?
- What are the components of GitHub Actions?
- What is a Runner?

★ Final Interview Gold

1. - GitHub Actions
2. - Workflow
3. - Runner
4. - Playwright Integration
5. - Enterprise Pipeline
6. - Workflow Execution
7. - GitHub Actions Best Practices
8. Jenkins

JENKINS

22. Jenkins ★

► What is Jenkins?

What is it: Jenkins is an open-source automation server used to automate the build, testing, and deployment of applications. It enables Continuous Integration (CI) and Continuous Delivery (CD) by automatically executing pipelines whenever code changes occur.

Key Components:

- Jenkins Server
- Jobs
- Pipelines
- Plugins
- Agents
- Build History

How YOU should answer: Jenkins is an open-source CI/CD automation tool that automates building, testing, reporting, and deployment. It integrates with source control systems like Git and automatically executes pipelines whenever code changes are pushed.

Example

Developer Pushes Code

↓

Git Repository

↓

Jenkins Triggered

↓

```
Run Playwright Tests
```

```
↓
```

► Why Jenkins is Important

What is it: Jenkins automates repetitive tasks, providing continuous feedback and reducing manual effort.

Key Components:

- Automation
- Continuous Testing
- Faster Releases
- Report Publishing
- Notification Integration

How YOU should answer: Jenkins enables automated software delivery by continuously executing builds and automation tests, improving software quality and reducing manual intervention.

```
Example
```

```
Developer Commit
```

```
↓
```

```
Jenkins Build
```

```
↓
```

```
Smoke Tests
```

```
↓
```

```
Results Ready
```

Common Mistakes

- ✗ Manual execution of automation.
- ✗ No automated reporting.

Expected Interview Questions

- What is Jenkins?
- Why do we use Jenkins?
- Why is Jenkins important?
- What is the difference between Master and Agent?
- Freestyle vs Pipeline?

★ Final Interview Gold

1. - Jenkins
2. - Jenkins Architecture
3. - Master vs Agent
4. - Pipeline

5. - Playwright Integration
6. - Enterprise Jenkins Pipeline
7. - Jenkins Best Practices
8. Docker

DOCKER

23. Docker ★

► What is Docker?

What is it: Docker is a containerization platform that packages an application along with all its dependencies into a lightweight, portable container. Containers ensure the application runs consistently across different environments such as developer machines, QA servers, and CI/CD pipelines.

Key Components:

- Docker Engine
- Docker Image
- Docker Container
- Docker Registry
- Dockerfile
- Docker Hub

How YOU should answer: Docker is a containerization platform that packages applications and their dependencies into isolated containers, ensuring consistent execution across different environments.

Example

Developer Machine

↓

Docker Image

↓

Docker Container

↓

QA Server

↓

► Why Docker is Important

What is it: Docker eliminates the "works on my machine" problem by ensuring the same execution environment everywhere.

Key Components:

- Environment Consistency

- Portability
- Faster Deployment
- Lightweight
- Isolation

How YOU should answer: Docker provides a consistent execution environment, reducing environment-related issues and making automation more reliable across development, testing, and production.

Example

Local Machine

↓

Docker

↓

Jenkins

↓

GitHub Actions

↓

Common Mistakes

- ✗ Confusing Docker with Virtual Machines.
- ✗ Assuming Docker replaces CI/CD.

Expected Interview Questions

- What is Docker?
- Why do we use Docker?
- Why is Docker important?
- Image vs Container?
- What is Dockerfile?

★ Final Interview Gold

1. - Docker
2. - Docker Architecture
3. - Image vs Container
4. - Playwright Integration
5. - Enterprise Docker Workflow
6. - Docker Components
7. - Docker Best Practices
8. Framework Best Practices

FRAMEWORK BEST PRACTICES

24. Framework Best Practices ★

► What are Framework Best Practices?

What is it: Framework Best Practices are a set of standards, design principles, and coding guidelines that make an automation framework scalable, maintainable, reusable, reliable, and easy to extend. A good framework should allow new test cases and features to be added with minimal effort while keeping execution

How YOU should answer: Framework Best Practices are design principles that ensure automation frameworks remain scalable, maintainable, reusable, and reliable as projects grow.

Example

Well Structured Framework

↓

Easy Maintenance

↓

Reusable Components

↓

Stable Automation

↓

Expected Interview Questions

- What are Framework Best Practices?
- Why are they important?
- Why should automation frameworks follow standards?
- How do you organize your framework?
- How do you improve framework maintainability?

★ Final Interview Gold

1. - Framework Architecture
2. - Project Structure
3. - Reusability
4. - Maintainability
5. - Smart Wait Strategies
6. - Parallel Execution

7. - Cross-Browser Execution

8. - CI/CD Integration

Kalpesh Jain | SDET @ Algebrik AI | [linkedin.com/in/kalpeshnjain09](https://www.linkedin.com/in/kalpeshnjain09) | Part 2 of 2